

model2rtl: Compiling a Quantized Neural Network into Portable RTL with a Shared Multiply-Select-Add Fabric

Rithwik

Independent Research

Email: sniyaantony@gmail.com

Abstract—Deploying a neural network as dedicated hardware normally means instantiating one multiplier per synapse, or accepting the area and control cost of a general matrix engine. We describe *model2rtl*, a compiler that emits portable, synthesizable Verilog-2001 for a quantized multilayer perceptron using a *Multiply-Select-Add* (MSA) fabric. Because every weight is one of $K=16$ fixed integer levels, an activation x_i can participate in only 16 distinct products regardless of its fanout. The fabric computes those 16 products once per cycle and lets every output neuron select the one its 4-bit weight index names. Execution is input-serial and output-parallel, so a single 16-entry product bank serves both layers of the network. The design separates the compute fabric from the trained parameters completely: regenerating the fabric from a different weight set yields a byte-identical file, which we verify on every compilation. We validate a 784–32–10 MNIST network end to end. Quantization to 4-bit weight indices and uint8 activations costs 0.07 percentage points of test accuracy (96.52% $\rightarrow$ 96.45%). Over 500 test images the RTL reproduces a pure-integer reference model with zero mismatches across 16,000 hidden activations and 5,000 logits, supported by 178,840 cycle-level internal checks. The *same* source then synthesizes through an FPGA-oriented and a generic/ASIC-oriented Yosys flow with no source change, and both gate-level netlists reproduce the integer model exactly on the same 500 images. Neither netlist retains a single multiplier or DSP cell. We additionally generate seven SKY130 mask-ROM macros for the parameters and verify all 102,640 programmed cells against the canonical images, and we report a storage-area sweep in which the hard ROM does not beat synthesized standard-cell storage anywhere in the measured 1,024–50,176-bit range.

Index Terms—neural network accelerators, quantization, RTL generation, weight sharing, multiplier-free inference, open-source EDA

I. INTRODUCTION

A trained neural network is a fixed function. Once training stops, the weights never change again, and in principle nothing about the inference datapath needs to be general. This observation motivates a long line of work on turning networks directly into circuits [6]–[8].

The obstacle is arithmetic. A dense layer with n inputs and m outputs implies nm multiplications. Building them spatially is prohibitive for all but the smallest networks; time-multiplexing them onto a smaller array reintroduces the con-

trol, buffering and scheduling machinery the direct approach was meant to avoid.

Aggressive weight quantization changes the shape of the problem. If every weight is drawn from a small alphabet of K integer levels, then for a given activation x_i there are only K distinct products $x_i \cdot a_k$ that can ever be required — no matter how many neurons x_i feeds. The multiplication is a property of the *activation*, not of the synapse. Each synapse then needs only to *select* among precomputed products, which costs a multiplexer rather than a multiplier. We refer to this arrangement as *Multiply-Select-Add*, a term that also appears in public trade coverage of commercial hardwired-inference silicon [11].

This paper contributes:

- 1) An RTL generator for a *Multiply-Select-Add* fabric that is *provably independent of the trained parameters*: substituting a different weight and bias set yields a byte-identical Verilog file, checked by SHA-256 on every compilation.
- 2) An integer arithmetic contract fixed analytically *before* any RTL exists, containing no multiplicative requantization scale, which is what makes that independence achievable.
- 3) A verification methodology in which a pure-NumPy integer model is the sole oracle at every level — behavioral simulation, post-synthesis gate-level simulation on two independent targets, and physical mask-ROM contents.
- 4) Measured results, including a negative one: the hard ROM we generate is larger than synthesized standard-cell storage at every size we measured.

We are explicit about scope. This work produces and verifies RTL. It does not perform place-and-route, timing analysis or a full physical implementation, and it makes no claim about clock frequency or finished-chip area.

II. RELATED WORK

Quantization. Jacob et al. [1] established integer-only inference with a multiplicative rescale between layers, which underpins TFLite quantization. Our contract deliberately omits that multiplier: the only requantization operator is a fixed

TABLE I

WHERE EACH APPROACH TAKES THE SAVING. ALL TARGET THE SAME BOTTLENECK: INFERENCE ARITHMETIC DOMINATES AREA AND ENERGY.

Work	Lever	Artifact
Deep Compression [2]	shared weights	compressed model
Jacob et al. [1]	integer-only math	quantized graph
DeepShift [4]	powers of two	shift kernels
UCNN [3]	run-time repetition	ASIC accelerator
LUTNet [5]	FPGA LUT as operator	LUT netlist
LogicNets [6]	neuron $\equiv$ truth table	LUT netlist
FINN [7]	binarized dataflow	FPGA bitstream
hls4ml [8]	HLS from model	FPGA firmware
This work	static K-level bank	portable Verilog

power-of-two shift. This is a restriction, but it is precisely what allows the fabric to contain no trained value, since a learned scale would otherwise have to be embedded in the datapath. Han et al. [2] introduced weight sharing via k -means clustering, reducing AlexNet storage $35\times$; their shared weights are a storage-side compression, whereas we exploit the same redundancy on the *compute* side.

Computational reuse. UCNN [3] is the closest prior work in spirit: it exploits weight repetition to reuse dot-product sub-computations, generalizing sparsity (repeated zeros) to repeated values, and reports $1.2\times$ – $4\times$ energy improvement over an Eyeriss-style baseline. UCNN discovers repetition dynamically in a general accelerator. We instead *fix* the alphabet at $K=16$ by construction, which makes the reuse structure static and lets it be baked into the RTL rather than detected at run time.

Multiplier-free inference. DeepShift [4] constrains weights to powers of two so multiplication becomes shift-and-sign-flip. Our alphabet $\{-8, \dots, +7\}$ is not restricted to powers of two, yet we observe the same outcome empirically: synthesis removes every multiplier anyway, because one operand is a small constant (Section VII-C).

LUT-native and circuit-level networks. LUTNet [5] treats the FPGA K -LUT itself as the inference operator, achieving roughly twice the area efficiency of binarized baselines. LogicNets [6] goes further, training quantized sparse networks whose neurons are directly equivalent to truth tables and emitting a netlist of LUTs. Both are strongly FPGA-shaped: the artifact is a LUT netlist. Our output is vendor-neutral Verilog-2001, which we demonstrate synthesizes unchanged through both an FPGA flow and a generic gate-level flow.

Toolflows. FINN [7] builds streaming FPGA dataflow accelerators for binarized networks, reporting 12.3M MNIST classifications/s on a ZC706. hls4ml [8] compiles small networks to FPGA firmware via HLS for sub-microsecond physics triggers. Both target FPGAs and rely on vendor-specific backends; both are considerably more capable than this work in network coverage. Our contribution is orthogonal: a single RTL source demonstrated portable across FPGA-oriented and ASIC-oriented synthesis, with post-synthesis functional equivalence verified on both.

Synthesis across these works shares a common motivation — inference arithmetic is the bottleneck and quantization is the lever. They diverge on *where* the saving is taken: in storage [2], in the operator [4], in the target’s native primitive [5], [6], or in run-time reuse [3]. The strongest results are FPGA-specific. The gap we address is portability with evidence: to our knowledge, prior work does not demonstrate one unmodified RTL source passing gate-level functional verification after two independent synthesis flows.

III. ARITHMETIC CONTRACT

The integer specification was fixed analytically before any RTL was written and never changed thereafter. Weight indices are 4 bits; the alphabet is the plain two’s-complement range $a_k = k - 8$, $k \in [0, 15]$, giving levels -8 to $+7$. Activations are unsigned 8-bit with zero point 0.

For layer 1, with $\mathbf{x} \in [0, 255]^{784}$:

$$\text{acc}_1[j] = \sum_i x_i \cdot a_{W_1[i][j]} + b_1[j] \quad (1)$$

$$h[j] = \text{clamp}\left(\frac{\max(\text{acc}_1[j], 0) + 128}{256}, 0, 255\right) \quad (2)$$

where the division is a round-half-up arithmetic right shift by 8. Layer 2 stops after the bias; the prediction is $\arg \max$ over signed logits, with ties broken by lowest index.

Widths are derived from the alphabet and layer sizes rather than assumed: products are 12-bit signed, layer-1 dot products 22-bit, layer-1 accumulators 23-bit, layer-2 dot products 17-bit and accumulators 18-bit.

The critical property of (2) is what it *lacks*. There is no multiplicative requantization scale anywhere in the datapath. The shift is an architectural constant, not a learned one. Consequently the fabric depends only on topology, K , the activation width and the shift — never on a trained value.

Quantization-aware training with straight-through estimators simulating the exact integer pipeline yields 96.45% MNIST test accuracy against a 96.52% float baseline, a loss of 0.07 percentage points. All 16 alphabet levels are used in both layers. Weight saturation during training is 0.92% in layer 1 and 7.5% in layer 2; the higher layer-2 figure reflects its 320 synapses and wider per-synapse dynamic range and does not degrade final accuracy.

IV. THE MULTIPLY-SELECT-ADD FABRIC

Fig. 1 shows the datapath. Execution is *input-serial, output-parallel*: one activation enters per cycle, the 16 products are formed combinatorially, and all neurons of the active layer accumulate in parallel.

Three operation counts must be distinguished, and conflating them is the most common way to overstate this kind of architecture:

The implemented count is 16 because the bank is recomputed each cycle for the current activation rather than unrolled across inputs. The cost is latency:

$$T = n_{\text{in}} + 2n_{\text{hid}} + n_{\text{out}} + 6 = 864 \text{ cycles} \quad (3)$$

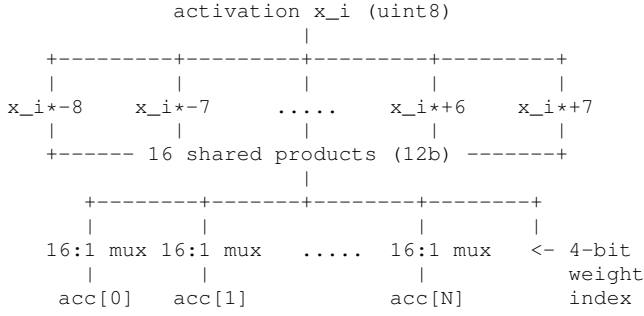


Fig. 1. One product bank of $K=16$ generators is shared by every output neuron of the active layer, and reused across input cycles and across both layers. Each synapse contributes a multiplexer, not a multiplier.

TABLE II
SOURCE-LEVEL OPERATION COUNTS FOR 784-32-10. NONE IS A PHYSICAL MULTIPLIER COUNT.

Organization	Products
Naive fully spatial (one per synapse)	25,408
Fully spatial MSA	13,056
Implemented (input-serial MSA)	16

per inference, data-independent. Elaborating the generated Verilog confirms the structure: exactly 16 `$mul` cells and 42 selector instances appear in the netlist, matching 32 layer-1 plus 10 layer-2 neurons.

Notably, sharing is not universally cheaper. It reduces product generators only when a layer’s output fanout exceeds K . Layer 1 (fanout 32) halves them; layer 2 (fanout 10) would need $1.6\times$ more in a fully spatial arrangement. The input-serial organization sidesteps this because the same 16 generators serve both layers.

A. When does sharing pay?

Sharing reduces product generators only when a layer’s output fanout exceeds K . For a layer with n inputs and m outputs the naive spatial count is nm while the fully spatial MSA count is nK , so sharing wins iff $m > K$. Layer 1 ($m=32 > 16$) halves its generators; layer 2 ($m=10 < 16$) would need $1.6\times$ more. The crossover sits exactly at fanout $= K$.

The input-serial organization removes this concern entirely, because the same 16 generators are reused across input cycles *and* across both layers: the implemented count is K regardless of either layer’s fanout. This is a practical argument for time-multiplexing beyond area alone — it makes the architecture insensitive to layer shape.

The premise also requires that the alphabet actually be used. Table III confirms it: all 16 levels are populated in both layers, with the expected concentration near zero.

B. Weight independence

The fabric is generated from topology alone. To verify this rather than assert it, the compiler regenerates the fabric from a random parameter set of the same shape on every

TABLE III
WEIGHT-INDEX DISTRIBUTION AFTER QUANTIZATION-AWARE TRAINING (REPRESENTATIVE LEVELS SHOWN). ALL 16 LEVELS ARE POPULATED IN BOTH LAYERS.

Level	-8	-6	-4	-2	0	+2	+4	+7
Layer 1	228	312	1013	2572	4201	2868	997	253
Layer 2	23	8	18	16	22	28	26	8

run and compares SHA-256. For the reference model the fabric hashes to `7757362642b37fd0...` under the trained weights, under a substituted weight set, and under substituted biases. Compilation fails closed if these ever differ. Only the parameter ROM is model-dependent.

V. PARAMETER STORAGE

Parameters are packed into four canonical images: weights 784×128 and 32×40 bits (eight 4-bit indices per 32-bit group, neuron 0 in the least significant field), and biases 32×22 and 10×17 signed. These images are hashed and are the single source of truth for every backend, which makes it structurally impossible to physically build one dataset while testing another.

Two backends implement the same fixed interface — synchronous read, one-cycle latency, enable-gated hold — so the fabric cannot tell which is attached: (i) a portable Verilog-2001 case/constant ROM usable on both FPGA and ASIC, and (ii) SKY130 mask-ROM macros generated with OpenRAM’s ROM compiler [9].

VI. VERIFICATION METHODOLOGY

A pure-NumPy integer model implementing the contract of Section III is the sole oracle at every stage. Floating-point Keras output is a reference number only and is never used to check RTL arithmetic. Verification proceeds at four levels: behavioral RTL, post-synthesis gate level on two targets, physical ROM contents, and full-model inference through the physical organization.

VII. RESULTS

A. Behavioral verification

Over the first 500 images of the official MNIST test set, taken in order with no filtering, the RTL reproduces the integer model exactly: 0 mismatches across 16,000 hidden activations, 5,000 logits and 500 predictions, on both backends and at 864 cycles every image. Cycle-level internal checkpointing over 20 images contributes 178,840 further comparisons with 0 failures, including per-cycle proof that each consumed ROM word corresponds to the address issued exactly one cycle earlier. Stall, reset, back-to-back, argmax-tie and arithmetic-edge suites are all clean.

Input timing was varied to confirm the handshake is not accidentally order-dependent. With no stalls an inference takes 864 cycles; with a bubble after every seventh accepted activation, 975 cycles; under a pseudo-random LFSR pattern, 1,223–1,297 cycles. Results are bit-identical in all three cases

TABLE IV
VERIFICATION LEVELS. THE SAME PURE-INTEGER MODEL IS THE ORACLE THROUGHOUT.

Level	Comparisons	Mismatches
Behavioral RTL, 500 images	21,500	0
Cycle-level internal trace, 20 images	178,840	0
Post-synthesis GLS, iCE40	5,500	0
Post-synthesis GLS, generic	5,500	0
Physical ROM programmed cells	102,640	0
Weight indices via physical banks	25,408	0

and only latency changes. Reset was injected at six points with no stale state surviving, and 500 inferences ran back to back in one simulation with no reset between them.

B. Dual-target synthesis portability

The same four RTL files, byte-identical, were synthesized through `synth_ice40` and through an independent generic flow (`proc/flatten/opt/memory/techmap/abc -g simple`) using Yosys [10]. No source was patched. Both netlists were then gate-level simulated with the official cell models.

TABLE V
POST-SYNTHESIS GATE-LEVEL RESULTS, 500 MNIST IMAGES PER TARGET.

	iCE40	Generic
Logits compared	5,000	5,000
Logit mismatches	0	0
Prediction mismatches	0	0
Cycles per inference	864	864
Total cells	9,079	45,707
LUT4 / gates	6,429	40,614
Flip-flops	1,614	1,742
Carry cells	1,004	—
Block RAMs	32	0
Multiplier / DSP cells	0	0

Table V summarizes. The two netlists also agree with each other bit for bit. Both flows are byte-deterministic across clean re-runs.

Two observations deserve emphasis. First, the portable case-ROM was *automatically* inferred into 32 iCE40 block RAMs with no FPGA-specific RTL, vendor primitive or synthesis pragma. Second, the architectural latency of 864 cycles survived both flows: synthesis added no pipeline stage.

C. What synthesis did to the multiplications

The source contains exactly 16 $\times$ operators. Both netlists contain *zero* multiplier or arithmetic cells. Inspecting the netlists rather than assuming, of the 192 product-bank bits, 37 (FPGA) are literal constants and 41 are plain aliases of the activation register — 41% of the bank costs no logic at all. Where the FPGA flow preserved wire names, the drivers are explicit: $x \cdot 0$ folded to a literal zero, $x \cdot 1$, $x \cdot 2$ and $x \cdot 4$ became pure wiring with constant zero fill, and $x \cdot (-8)$ is a shift of a shared negated value. The remaining 114 bits have no driver at all: synthesis fused product generation into the 16:1 selection.

The honest statement is therefore not that we reduced 25,408 multipliers to 16. It is that *the architecture exposes only sixteen constant-weight product alternatives per activation, and synthesis then eliminates explicit multiplier hardware entirely*. The three counts in Table II are source-level quantities; the synthesized results are a different kind of measurement.

D. Physical ROM generation

Seven SKY130 macros were generated. Two physical transformations were needed because the ROM compiler expresses word size in bytes and could not route a 784×128 array: layer-1 weights were banked into four parallel 784×32 macros sharing one address, and both bias memories were stored as 24-bit sign-extended words. Both transformations are exactly reversible, and the reverse map is an automated invariant.

Every programmed cell was read back out of the generated SPICE netlists and compared against the canonical images: **102,640 / 102,640 exact**. All 858 logical rows reconstruct from the banks and all 25,408 weight indices survive. The physical organization has zero functional effect: over the same 500 images the physical backend gives 0 hidden, logit and prediction mismatches at 864 cycles.

E. Storage area

TABLE VI
STORAGE-SIZE SWEEP. IDENTICAL DETERMINISTIC CONTENTS; OPENROM AREA IS A GDS BOUNDING BOX, PORTABLE AREA IS A LIBERTY CELL-AREA SUM. THESE ARE DIFFERENT KINDS OF AREA.

Bits	OpenROM (μm^2)	Portable (μm^2)	Ratio
1,024	13,275	1,964	6.76
4,096	18,467	4,773	3.87
16,384	38,879	13,753	2.83
25,088	53,669	18,434	2.91
50,176	92,011	30,935	2.97

The seven macros total $252,382 \mu\text{m}^2$ of GDS bounding box. The same parameters mapped to SKY130 standard cells occupy 9,406 cells and $58,336 \mu\text{m}^2$ of liberty cell area, a raw ratio of $4.33\times$.

Table VI sweeps storage size. **No crossover was measured:** the portable mapping is smaller at every point from 1,024 to 50,176 bits, and the ratio flattens near $2.9\times$ rather than converging toward 1. We state this narrowly. It does not establish that a hard ROM never wins; it establishes that no OpenROM area advantage was observed over the measured range with this generator and library. As context only — not a measurement — the two would break even at the deepest point if the portable block were placed at 33.6% utilization or worse.

VIII. GENERALIZATION BEYOND THE REFERENCE NETWORK

The architecture is not specific to 784–32–10. The compiler accepts any two-layer dense network: it infers the topology from the model, derives all datapath widths from the alphabet and layer sizes, and emits the corresponding fabric. To test this

on a model the project had never seen, we trained a 784–64–10 network on Fashion-MNIST and compiled it.

Post-training quantization alone is not free at 4 bits, because the float weights were never trained to survive a 16-level alphabet. Fine-tuning with the exact integer pipeline in the forward pass recovers the loss and more. Table VII reports accuracy on a held-out split the quantizer never saw.

TABLE VII
FASHION-MNIST 784–64–10, HELD-OUT SPLIT. RTL WAS SIMULATED,
NOT MERELY ELABORATED.

	Accuracy
Float Keras baseline	83.0%
Post-training quantization → RTL	82.7%
Quantization-aware fine-tuning → RTL	86.0%
RTL vs. integer model (3,000 logits)	0 mismatches
Cycles per inference	928

Two details matter. The requantization shift is no longer an architectural constant once the topology is free: it must be chosen per model. We choose it by measurement, sweeping candidate shifts against calibration data. The sweep exposes a clear trade — at shift 4, 43% of hidden activations saturate and accuracy collapses to 74.8%; at shift 12, 48% are driven to zero and accuracy falls to 96.2%; the plateau sits at 7–9. Assuming a value here would silently cost several points.

The compiler also refuses what it cannot build. A three-layer MLP and a convolutional model are rejected with the layers they actually contain, rather than partially compiled. In one downloaded case a three-layer network had dimensionally compatible first and last layers, so dropping the middle layer would have produced hardware that elaborates cleanly and computes a different network. Refusing is the correct behavior.

IX. DISCUSSION

The enabling decision in this work is arithmetic, not architectural. Fixing the requantization operator to a power-of-two shift, and admitting no multiplicative scale, is what makes the compute fabric provably free of trained values. Had we adopted the more conventional integer pipeline with a learned rescale [1], a trained constant would necessarily appear in the datapath and the clean separation between fabric and parameter image would be lost. The restriction costs accuracy — 0.07 points here — and buys a property that is easy to verify mechanically and hard to lose by accident.

The synthesis result reinforces this. We wrote 16 multiplications and got zero multiplier cells on two independent targets. The architecture’s job was not to avoid multipliers directly but to arrange the arithmetic so that a standard synthesis tool could remove them. That suggests a general design heuristic for small quantized networks: expose constant operands to the tool and let it do the strength reduction, rather than hand-coding shift-and-add.

The negative area result deserves to be carried forward rather than buried. Over 1,024–50,176 bits an open-source

mask ROM was consistently larger than synthesized standard-cell storage, and the ratio flattened near $2.9\times$ rather than converging. We cannot attribute this: it may reflect the generator’s peripheral overhead, the library, or the size range, and distinguishing those would require either a denser ROM generator or measurements well beyond the sizes we could build. What the data does support is a caution — the intuition that a hard ROM is obviously denser than synthesized logic did not hold for any size we could measure.

X. FUTURE WORK

The most valuable extensions, in rough order of leverage: importing models from interchange formats such as ONNX rather than a single framework; supporting arbitrary depth, which requires generalizing the two-phase control FSM and the one-bit layer field in the memory interface; convolution lowering; and place-and-route with timing analysis on both targets, which is the missing evidence needed before any performance claim could be made. A trustworthy physical-verification environment would also be required before the mask-ROM path could be taken seriously.

XI. LIMITATIONS

We are deliberate about what is not demonstrated.

No physical signoff. DRC and LVS results in our environment are not trustworthy: OpenRAM’s *own* upstream reference ROM fails here with 830 DRC errors and an LVS mismatch. Consequently no result we produced is evidence about our macros in either direction, and no macro is called DRC- or LVS-clean. Physical *generation* passes; physical *signoff* is UNVERIFIED. These are separate verdicts and we report them separately.

No place-and-route or timing. Neither target was placed, routed or timed. No clock frequency is claimed; cycle counts are exact but any MHz figure would be fabricated. The area comparison is between a hard-macro bounding box and a standard-cell area sum, and is not a finished-block ratio.

Scope. MNIST only, one topology family, two dense layers, 4-bit weights and uint8 activations. No convolution, no ONNX/TFLite ingestion. The banking scheme is specific to the demonstrated ROM shape.

Prior art. This work explores a digital RTL interpretation of publicly disclosed high-level Multiply-Select-Add ideas associated with public trade coverage of commercial hardwired-inference silicon [11]. That source is industry press, not peer-reviewed, and we grade it accordingly. No proprietary source code, netlist, layout or transistor-level mask-ROM detail was used, consulted or reproduced, and nothing here is claimed to be equivalent to that hardware.

XII. CONCLUSION

model2rtl demonstrates that a quantized MLP can be represented as a portable, weight-index-driven RTL compute fabric whose functional behavior remains bit-exact across behavioral simulation, FPGA-oriented synthesis and generic logic synthesis. The enabling design decision is arithmetic,

not architectural: by fixing the requantization operator to a power-of-two shift and admitting no multiplicative scale, the compute fabric becomes provably free of trained values, and the model reduces to a parameter image behind a fixed interface. The measured outcome — zero multiplier cells after synthesis on both targets, and zero functional mismatches on 500 images at every level of verification — suggests that for small quantized networks the interesting question is not how to build multipliers efficiently, but how to arrange the arithmetic so that synthesis can remove them.

The negative area result is worth carrying forward. Over the sizes we could measure, an open-source mask ROM did not beat synthesized standard-cell storage. Whether that reflects the generator, the library, or the size range is the natural next question.

REPRODUCIBILITY AND AI DISCLOSURE

All RTL, parameter images, per-stage JSON reports and the full test suite are part of the artifact. Every number in this paper was extracted programmatically from those reports rather than transcribed. Quantities recorded by more than one stage were cross-checked rather than reconciled. AI-assisted tools were used in the development, verification and preparation of this work; all reported measurements were produced by the named EDA tools and are reproducible from the artifact.

REFERENCES

- [1] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, “Quantization and training of neural networks for efficient integer-arithmetic-only inference,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 2704–2713.
- [2] S. Han, H. Mao, and W. J. Dally, “Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2016.
- [3] K. Hegde, J. Yu, R. Agrawal, M. Yan, M. Pellauer, and C. W. Fletcher, “UCNN: Exploiting computational reuse in deep neural networks via weight repetition,” in *Proc. 45th ACM/IEEE Int. Symp. Computer Architecture (ISCA)*, 2018, pp. 674–687.
- [4] M. Elhoushi, Z. Chen, F. Shafiq, Y. H. Tian, and J. Y. Li, “DeepShift: Towards multiplication-less neural networks,” arXiv:1905.13298, 2019.
- [5] E. Wang, J. J. Davis, P. Y. K. Cheung, and G. A. Constantinides, “LUTNet: Learning FPGA configurations for highly efficient neural network inference,” *IEEE Trans. Computers*, vol. 69, no. 12, pp. 1795–1808, 2020.
- [6] Y. Umuroglu, Y. Akhauri, N. J. Fraser, and M. Blott, “LogicNets: Co-designed neural networks and circuits for extreme-throughput applications,” in *Proc. 30th Int. Conf. Field-Programmable Logic and Applications (FPL)*, 2020.
- [7] Y. Umuroglu, N. J. Fraser, G. Gambardella, M. Blott, P. Leong, M. Jahre, and K. Vissers, “FINN: A framework for fast, scalable binarized neural network inference,” in *Proc. ACM/SIGDA Int. Symp. Field-Programmable Gate Arrays (FPGA)*, 2017, pp. 65–74.
- [8] J. Duarte *et al.*, “Fast inference of deep neural networks in FPGAs for particle physics,” *Journal of Instrumentation*, vol. 13, no. 07, P07027, 2018.
- [9] M. R. Guthaus, J. E. Stine, S. Ataei, B. Chen, B. Wu, and M. Sarwar, “OpenRAM: An open-source memory compiler,” in *Proc. 35th Int. Conf. Computer-Aided Design (ICCAD)*, 2016.
- [10] C. Wolf, J. Glaser, and J. Kepler, “Yosys — a free Verilog synthesis suite,” in *Proc. 21st Austrian Workshop on Microelectronics (Austrochip)*, 2013.
- [11] T. P. Morgan, “Taalas etches AI models onto transistors to rocket boost inference,” *The Next Platform*, Feb. 19, 2026. [Industry press; not peer-reviewed.]